CST1510 Programming for data communication and networks

Week 6: Example of text based Shopping System project implementation

WORKSHOP

Shopping System Project Overview

Scenario: Imagine a simple **Shopping System** that allows users to browse products, add them to a cart, and complete purchases. This system offers a familiar shopping experience, providing a text-based interface where users can explore items, adjust cart quantities, and checkout. Over the course of the project, the system will evolve from a basic text structure to an OOP-based application with client-server communication, database integration, and an eventual graphical interface.

Project Goals: The project models a client-server shopping platform. The client handles the user interface, product selection, and checkout, while the server manages product inventory, order processing, and transaction records.

1. **Product Viewing and Selection:** Display available products with details on prices and stock.
2. **Cart Management:** Add items to a virtual cart, manage quantities, and view a cart summary.
3. **Checkout Simulation:** Confirm purchase and update stock after successful checkout.
4. **Order History:** Store transaction history and display past orders on request.
5. **Automatic Stock Update:** Server updates inventory after each purchase.

Instructions Step by step

Step 1: Set Up the Inventory File

1. **Create the `inventory.txt` File**:
   - Open a text editor and save a file named `inventory.txt` in the project directory.
   - Add sample product data in CSV format, with each line representing a product: `ID, Product Name, Price, Stock`.

- Example:
  plaintext
  1, Sneakers, 49.99, 10
  2, Backpack, 29.99, 15
  3, Water Bottle, 9.99, 20

Step 2: Implement `load_inventory()`

Goal: Load product data from `inventory.txt` into a list of dictionaries.

1. **Create the `load_inventory()` function**:
   - Open a new Python file (e.g., `shopping_system.py`) and define the `load_inventory()` function.
   - This function reads `inventory.txt` and returns a list of product dictionaries, each containing `id`, `name`, `price`, and `stock`.

In [35]:
```
inventory = {}

# Path to your file
path = "inventory.txt"

# Open the file and read its content
with open(path, "r") as file:
    data = file.readlines()

# Get the headers from the first line, stripping any whitespace
keys = data[0].strip().split(",")

# Initialize each key in the dictionary with an empty list
for key in keys:
    inventory[key] = []

# Loop through the rest of the lines and populate the dictionary
for line in data[1:]:
    values = line.strip().split(",")
    # Pair each key with its corresponding value and append to the dictio
    for key, value in zip(keys, values):
        inventory[key].append(value)

# Print the resulting dictionary to verify
print(inventory)
```

```
{'Id': ['1', '2', '3'], ' Product_Name': [' Sneakers', ' Backpack', ' Wate
r Bottle'], ' Price': [' 49.99', ' 29.99', ' 9.99'], ' Stock': [' 10', ' 1
5', ' 20']}
```

Step 3: Implement `view_products()`

Goal: Display all available products with details.

1. **Create the `view_products()` function**:
   - This function will loop through `inventory` and print each product's ID, name, price, and stock.

In [36]:
```
def view_products(inventory):
    print("\nAvailable Products:")
    # Loop through each product by index
    for i in range(len(inventory['Id'])):
        # Access each product's details by index `i`, using .strip() to r
        print(f"ID: {inventory['Id'][i].strip()} - {inventory[' Product_N



view_products(inventory)
```

```
Available Products:
ID: 1 - Sneakers - $49.99 - Stock: 10
ID: 2 - Backpack - $29.99 - Stock: 15
ID: 3 - Water Bottle - $9.99 - Stock: 20
```

Step 4: Set Up Cart and Implement `add_to_cart()`

Goal: Create a shopping cart and allow users to add products to it.

1. **Initialize an empty cart**:
   - Define an empty dictionary named `cart` at the top of your file.
2. **Create the `add_to_cart()` function**:
   - The function takes `product_id`, `quantity`, and `inventory` as parameters.
   - It verifies if the product exists and checks stock availability. If sufficient, it adds to `cart` and reduces stock.

Step 4: Set Up Cart and Implement `add_to_cart()`

Goal: Create a shopping cart system that allows users to add products to the cart based on product availability.

---

1. **Initialize an Empty Cart**:
   - At the top of your file, create an empty dictionary named `cart` to hold selected items.
   - This dictionary will store products by their IDs, where each product's quantity is updated as the user adds items.

In [37]:
```
cart = {}  # Initialize an empty cart
```

2. **Create the `add_to_cart()` Function**:
   - The function should take three parameters: `product_id`, `quantity`, and `inventory`.
   - **Process**:
     - First, it checks if the `product_id` exists in `inventory` by searching in `inventory['Id']`.

- If the product exists, it checks if enough stock is available by comparing the requested `quantity` with the available stock in `inventory[' Stock']`.
- If sufficient stock is available, the function does the following:
  - Adds the product to the cart, or if already present, updates the quantity.
  - Deducts the quantity from the product's stock in `inventory`.
- If stock is insufficient or the product doesn't exist, the function prints an error message.

In [38]:
```
def add_to_cart(product_id, quantity, inventory):
    # Check if product_id exists in inventory
    if product_id in inventory['Id']:
        index = inventory['Id'].index(product_id)

        # Convert stock and quantity to integers for comparison
        available_stock = int(inventory[' Stock'][index].strip())
        quantity = int(quantity)

        if available_stock >= quantity:
            # Add or update the product in the cart
            if product_id in cart:
                cart[product_id]['quantity'] += quantity
            else:
                cart[product_id] = {
                    'name': inventory[' Product_Name'][index].strip(),
                    'price': float(inventory[' Price'][index].strip()),
                    'quantity': quantity
                }

            # Update inventory stock
            inventory[' Stock'][index] = str(available_stock - quantity)
            print(f"Added {quantity} of {cart[product_id]['name']} to you
        else:
            print(f"Insufficient stock for {inventory[' Product_Name'][in
    else:
        print("Product ID not found.")
```

Explanation

1. **Check for Product Existence**:
   - `product_id in inventory['Id']`: Checks if the `product_id` is in the list of product IDs in the inventory.
2. **Find Product Index**:
   - `index = inventory['Id'].index(product_id)`: Finds the position of `product_id` in `inventory['Id']` to access other product details like `name`, `price`, and `stock`.
3. **Verify Stock**:
   - `available_stock >= quantity`: Confirms that enough stock is available.

4. **Add to Cart or Update Quantity**:
    - If the item is already in the cart, it updates the quantity. Otherwise, it adds a new entry with product details.
5. **Update Inventory Stock**:
    - Reduces the stock in `inventory[' Stock']` after successfully adding items to the cart.
6. **Error Handling**:
    - Prints a message if the `product_id` is not found or if there is insufficient stock.

In [39]:
```
#Example Usage
# Assuming `inventory` has been loaded and `cart` is empty:
# Example: Adding 2 Sneakers (product ID '1') to the cart

add_to_cart('1', 2, inventory)
```

```
Added 2 of Sneakers to your cart.
```

Step 5: Implement `view_cart()`

Goal: Display the current contents of the cart, including each item's name, quantity, unit price, and total price per item. At the end, display the total value of all items in the cart.

---

1. **Create the `view_cart()` Function**:
    - This function will loop through the `cart` dictionary, displaying details for each item.
    - For each item, it will print:
        - **Name**: The product name.
        - **Quantity**: The quantity added to the cart.
        - **Price per Unit**: The price of one unit of the product.
        - **Total Price**: Calculated as `quantity * price per unit` for that item.
    - Calculate the total value of all items in the cart and print it.

In [40]:
```
def view_cart():
    if not cart:
        print("\nYour cart is empty.")
        return

    print("\nYour Cart:")
    total_cost = 0  # Initialize total cost for all items in the cart

    # Loop through each item in the cart
    for product_id, item in cart.items():
        # Calculate total price for this item
        item_total = item['quantity'] * item['price']
        # Add to overall total cost
        total_cost += item_total
```

```
        # Print item details
        print(f"{item['name']}: Quantity: {item['quantity']} - Price per

    # Print total cost of the cart
    print(f"\nTotal Cart Value: ${total_cost:.2f}")
```

Explanation

1. **Check for Empty Cart**:
   - `if not cart`: Checks if the cart is empty. If so, it prints "Your cart is empty" and returns, skipping the rest of the function.
2. **Initialize `total_cost`**:
   - `total_cost = 0`: Starts with a total cost of zero, adding each item's total as we loop through the cart.
3. **Loop Through `cart`**:
   - `for product_id, item in cart.items()`: Iterates through each product in the `cart`, where `product_id` is the key and `item` is a dictionary containing the product's details.
4. **Calculate and Print Item Details**:
   - `item_total = item['quantity'] * item['price']`: Calculates the total cost for the quantity of each item in the cart.
   - Accumulates `item_total` to `total_cost`.
   - Prints the name, quantity, price per unit, and total for each item.
5. **Print Total Cart Value**:
   - After looping through all items, prints the `total_cost` as the overall cart value.

In [54]:
```
### Example Usage
# Assuming `cart` has some items:

cart = {
    '1': {'name': 'Sneakers', 'price': 49.99, 'quantity': 2},
    '2': {'name': 'Backpack', 'price': 29.99, 'quantity': 1}
}

view_cart()
```

```
Your Cart:
Sneakers: Quantity: 2 - Price per Unit: $49.99 - Total: $99.98
Backpack: Quantity: 1 - Price per Unit: $29.99 - Total: $29.99

Total Cart Value: $129.97
```

Step 6: Implement `checkout()`

Goal: Complete the checkout process by confirming the purchase, clearing the cart, and updating the inventory file to reflect changes in stock.

1. **Create the `checkout()` Function**:
   - The function will display the cart contents and total value using `view_cart()`.
   - It will then prompt the user to confirm the purchase.
   - If confirmed, the function will:
     - Call `save_inventory()` to update the inventory file with the new stock levels.
     - Call `save_transaction()` to record the completed transaction.
     - Clear the cart to finalize the checkout process.
   - If not confirmed, it simply returns without making changes.

In [55]:
```
def checkout():
    # Display cart contents
    view_cart()

    # Check if cart is empty before proceeding
    if not cart:
        print("\nYour cart is empty. Nothing to checkout.")
        return

    # Ask for confirmation to proceed with the checkout
    confirm = input("\nDo you want to proceed with the purchase? (yes/no)

    if confirm == 'yes':
        # Save the updated inventory and the transaction
        save_inventory(inventory)
        save_transaction(cart)

        # Clear the cart after checkout
        cart.clear()

        print("\nPurchase confirmed. Thank you for shopping with us!")
    else:
        print("\nCheckout cancelled. You can continue shopping.")
```

Explanation

1. **Display Cart Contents**:
   - Calls `view_cart()` to show the current items in the cart and total cost.
2. **Check for Empty Cart**:
   - If `cart` is empty, it prints a message and exits the function since there's nothing to check out.
3. **Confirm Purchase**:
   - Prompts the user with `"Do you want to proceed with the purchase? (yes/no): "`. If the user types `"yes"`, it proceeds; otherwise, it cancels checkout.
4. **Save Inventory and Transaction**:

- If confirmed, it calls `save_inventory()` to update the inventory file with adjusted stock.
- Then, it calls `save_transaction()` to log the transaction details.

5. **Clear the Cart**:
   - Empties `cart` to complete the checkout, indicating the purchase is finalized.
6. **Display Confirmation Message**:
   - Prints a message confirming or cancelling the checkout.

Helper Functions

- **`save_inventory()`**: This function will update the `inventory.txt` file with the latest stock levels after checkout.
- **`save_transaction()`**: This function will log each purchase in a `transactions.txt` file, recording what items were bought and in what quantities.

In [56]:
```
### Implementing `save_inventory()`

def save_inventory(inventory):
    with open("inventory.txt", "w") as file:
        # Write headers
        file.write("Id, Product_Name, Price, Stock\n")

        # Write each product's updated stock
        for i in range(len(inventory['Id'])):
            file.write(f"{inventory['Id'][i]},{inventory[' Product_Name']

save_inventory(inventory)
```

Explanation of Helper Functions

- **`save_inventory()`**:
  - Opens `inventory.txt` in write mode and writes the updated inventory data to the file.
  - Writes headers at the top, then iterates through each product in the inventory, writing its updated stock level.

In [59]:
```
### Implementing `save_transaction()`

def save_transaction(cart):
    with open("transactions.txt", "a") as file:
        file.write("Transaction:\n")
        for product_id, item in cart.items():
            file.write(f"{item['name']}, Quantity: {item['quantity']}, To
        file.write("\n")
save_transaction(cart)
```

Explanation of Helper Functions

- `save_transaction()`:
  - Opens `transactions.txt` in append mode to keep a log of each completed transaction.
  - For each item in the cart, it logs the product name, quantity, and total cost.

In [ ]:
```
### Step 7: Implement `save_inventory()` and `save_transaction()`

**Goal**: Implement functions to save the updated inventory back to `inve

---

1. **Create the `save_inventory()` Function**:
   - This function will write the current inventory data to `inventory.tx

2. **Create the `save_transaction()` Function**:
   - This function will log each completed transaction by writing details
```

In [ ]:
```
def save_inventory(inventory):
    # Open the file in write mode to overwrite with updated inventory
    with open("inventory.txt", "w") as file:
        # Write the header line
        file.write("Id,Product_Name,Price,Stock\n")

        # Write each product's details to the file
        for i in range(len(inventory['Id'])):
            # Get each product's details
            product_id = inventory['Id'][i]
            product_name = inventory[' Product_Name'][i].strip()
            price = inventory[' Price'][i].strip()
            stock = inventory[' Stock'][i].strip()

            # Write the product details in the correct format
            file.write(f"{product_id},{product_name},{price},{stock}\n")
    print("Inventory has been updated.")
```

Explanation of `save_inventory()`

1. **Open File in Write Mode**:
   - Opens `inventory.txt` in write mode, which overwrites the existing file contents with the updated inventory data.
2. **Write Header**:
   - Adds the header line to the file: `"Id,Product_Name,Price,Stock"`.
3. **Write Product Details**:
   - Loops through each product in the `inventory` dictionary.
   - For each product, retrieves `product_id`, `product_name`, `price`, and `stock`.
   - Writes these details in CSV format, one line per product.

In [60]:
```
### `save_transaction()` Implementation
### The `save_transaction()` function logs the transaction details to a f
```

```
def save_transaction(cart):
    # Open the file in append mode to add new transactions without erasin
    with open("transactions.txt", "a") as file:
        file.write("New Transaction:\n")

        # Write each item in the cart to the transactions file
        for product_id, item in cart.items():
            name = item['name']
            quantity = item['quantity']
            total_price = item['quantity'] * item['price']

            # Write each product's transaction details
            file.write(f"{name}, Quantity: {quantity}, Total: ${total_pri

        # Separate transactions with a line break for readability
        file.write("\n")
    print("Transaction has been recorded.")
```

Explanation of `save_transaction()`

1. **Open File in Append Mode**:
   - Opens `transactions.txt` in append mode, which adds new transaction records without overwriting existing ones.
2. **Write Transaction Header**:
   - Writes a header `"New Transaction:"` to mark the beginning of each new transaction.
3. **Log Each Item in Cart**:
   - Loops through each item in the `cart` dictionary.
   - For each item, retrieves its `name`, `quantity`, and `total_price`.
   - Writes these details in a readable format.
4. **Separate Transactions**:
   - Adds a line break at the end of each transaction for readability.

In [63]:
```
### Example Usage
# After a purchase is confirmed in `checkout()`, both `save_inventory()`
# If the `inventory` and `cart` look like this before checkout:


inventory = {
    'Id': ['1', '2', '3'],
    ' Product_Name': [' Sneakers', ' Backpack', ' Water Bottle'],
    ' Price': [' 49.99', ' 29.99', ' 9.99'],
    ' Stock': [' 8', ' 14', ' 20']
}

cart = {
    '1': {'name': 'Sneakers', 'price': 49.99, 'quantity': 2},
    '2': {'name': 'Backpack', 'price': 29.99, 'quantity': 1}
}
```

```
save_inventory(inventory)
save_transaction(cart)
```

```
Transaction has been recorded.
```

```
After running `save_inventory(inventory)` and
`save_transaction(cart)`, the `inventory.txt` file would
contain:

```plaintext
Id,Product_Name,Price,Stock
1,Sneakers,49.99,8
2,Backpack,29.99,14
3,Water Bottle,9.99,20
```
```

And `transactions.txt` would include:

```
plaintext
New Transaction:
Sneakers, Quantity: 2, Total: $99.98
Backpack, Quantity: 1, Total: $29.99
```

These functions finalize the checkout process, ensuring that inventory is updated and a record of each transaction is saved.

Step 8: Implement `menu()` to Combine All Functions

Goal: Create a user-friendly menu interface to allow navigation through the shopping system's features, such as viewing products, adding items to the cart, viewing the cart, and checking out.

---

1. **Create the `menu()` Function**:
   - The `menu()` function will display a list of options to the user.
   - Based on the user's choice, it will call the appropriate function to perform actions such as viewing products, adding items to the cart, checking out, or exiting the program.
   - Use a loop to keep displaying the menu until the user decides to exit.

In [64]:
```
def menu():
    print("Welcome to the Shopping System!")

    while True:
        # Display the menu options
        print("\nPlease choose an option:")
        print("1. View Products")
        print("2. Add to Cart")
        print("3. View Cart")
        print("4. Checkout")
        print("5. Exit")

        # Get the user's choice
```

```
        choice = input("Enter your choice (1-5): ").strip()

        # Call the appropriate function based on user choice
        if choice == '1':
            view_products(inventory)  # Display available products
        elif choice == '2':
            product_id = input("Enter the Product ID to add to cart: ").s
            quantity = input("Enter quantity: ").strip()

            # Convert quantity to integer and call add_to_cart
            if quantity.isdigit():
                add_to_cart(product_id, int(quantity), inventory)
            else:
                print("Invalid quantity. Please enter a number.")

        elif choice == '3':
            view_cart()  # Display items in the cart
        elif choice == '4':
            checkout()   # Proceed with checkout
        elif choice == '5':
            print("Thank you for shopping with us! Goodbye.")
            break       # Exit the loop and end the program
        else:
            print("Invalid choice. Please enter a number from 1 to 5.")
```

Explanation

1. **Display Welcome Message**:
   - Prints a welcome message when the user first enters the system.
2. **Menu Loop**:
   - Uses a `while True` loop to continuously display the menu options until the user chooses to exit.
3. **Display Menu Options**:
   - Prints a list of options each time the menu is shown:
     - `1. View Products`: Calls `view_products(inventory)` to display available products.
     - `2. Add to Cart`: Prompts the user to enter a `product_id` and `quantity`, then calls `add_to_cart()` to add the specified item to the cart.
     - `3. View Cart`: Calls `view_cart()` to display the current cart contents and total cost.
     - `4. Checkout`: Calls `checkout()` to complete the purchase, update inventory, and clear the cart.
     - `5. Exit`: Exits the loop, ending the program with a farewell message.
4. **Choice Validation**:
   - Checks the user's input to ensure it matches one of the valid choices (1 to 5).
   - If the input is invalid, it prompts the user to enter a correct option.
5. **Call Functions Based on Choice**:

- Calls different functions depending on the user's choice:
  - **View Products**: Calls `view_products()` to show available products.
  - **Add to Cart**: Calls `add_to_cart()` if the user inputs a valid quantity.
  - **View Cart**: Calls `view_cart()` to display the cart.
  - **Checkout**: Calls `checkout()` to confirm the purchase.
  - **Exit**: Breaks the loop and ends the program.

In [65]:
```
###### Example Usage

menu()
```

```
Welcome to the Shopping System!

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 1

Available Products:
ID: 1 - Sneakers - $49.99 - Stock: 8
ID: 2 - Backpack - $29.99 - Stock: 14
ID: 3 - Water Bottle - $9.99 - Stock: 20

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 1

Available Products:
ID: 1 - Sneakers - $49.99 - Stock: 8
ID: 2 - Backpack - $29.99 - Stock: 14
ID: 3 - Water Bottle - $9.99 - Stock: 20

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 2
Enter the Product ID to add to cart: 3
Enter quantity: 3
Added 3 of Water Bottle to your cart.

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 3

Your Cart:
Sneakers: Quantity: 2 - Price per Unit: $49.99 - Total: $99.98
Backpack: Quantity: 1 - Price per Unit: $29.99 - Total: $29.99
Water Bottle: Quantity: 3 - Price per Unit: $9.99 - Total: $29.97

Total Cart Value: $159.94

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
```

```
Enter your choice (1-5): 4

Your Cart:
Sneakers: Quantity: 2 - Price per Unit: $49.99 - Total: $99.98
Backpack: Quantity: 1 - Price per Unit: $29.99 - Total: $29.99
Water Bottle: Quantity: 3 - Price per Unit: $9.99 - Total: $29.97

Total Cart Value: $159.94

Do you want to proceed with the purchase? (yes/no): yes
Transaction has been recorded.

Purchase confirmed. Thank you for shopping with us!

Please choose an option:
1. View Products
2. Add to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 5
Thank you for shopping with us! Goodbye.
```

In [ ]: